



Industry Protocol (Lab Report)

Submitted by:

Mavia Ahmed

Submitted to:

Sir Zia Ahmed Shah

Table of Contents

(Industry Protocol Lab)

GitHub Repository Link: <https://github.com/mavia369/USTP-Industry-Protocol-Labs.git>

Week#	Date	Lab Objective
01	16-JUN-26 18-JUN-26 19-JUN-26	Design and simulate an APB Slave (Simple register-mapped peripheral)
02	23-JUL-26 25-JUL-26 26-JUL-26	Implement a simple AHB-Lite master/slave and test read/write transactions.
03		
04		

Week 1 LAB Report

Lab Task:

Objective:

Design and simulate an APB slave (simple register-mapped peripheral).

Introduction:

The Advanced Peripheral Bus (APB) is a simple, low-power bus protocol developed by ARM as part of the AMBA architecture. It is commonly used to connect low-speed peripherals such as timers, UARTs, GPIOs, and configuration registers. In this experiment, an APB slave implementing a simple register-mapped peripheral is designed and simulated using SystemVerilog.

Register-mapped Peripheral:

A register-mapped peripheral is a hardware module whose internal registers are assigned specific memory addresses. A processor or bus master can access these registers by performing read and write operations using the assigned addresses. This method provides a simple interface for configuring and controlling hardware peripherals through memory-mapped I/O.

Design Implementation:

The APB slave was implemented in SystemVerilog following the APB protocol. The design consists of four 32-bit registers (reg0, reg1, reg2, and reg3) mapped to four different addresses.

Address	Register	Operation
0x00	reg0	Read/Write
0x04	reg1	Read/Write
0x08	reg2	Read/Write
0x0C	reg3	Read/Write

Design Steps:

1. Define all APB interface signals, including PCLK, PRESETn, PADDR, PWDATA, PRDATA, PSEL, PENABLE, and PWRITE.
2. Create four internal 32-bit registers to store data.
3. Implement reset logic to clear all registers when PRESETn is low.
4. Implement write logic to store PWDATA into the selected register during a valid APB write transaction (PSEL, PENABLE, and PWRITE asserted).
5. Implement combinational read logic to place the selected register value on PRDATA during a read transaction.
6. Set PREADY permanently high to indicate the slave is always ready.
7. Set PSLVERR permanently low since no error handling is implemented in this simple peripheral.

Test Bench:

The testbench verifies the functionality of the APB slave by generating the required APB control signals and performing both write and read operations.

Test Cases:

Test Cases	Description	Expected Result
Reset	Assert PRESETn = 0	All registers become 0
Write Register 0	Write 0x11111111 to address 0x00	reg0 = 0x11111111
Write Register 1	Write 0x22222222 to address 0x04	reg1 = 0x22222222
Write Register 2	Write 0x33333333 to address 0x08	reg2 = 0x33333333
Write Register 3	Write 0x44444444 to address 0x0C	reg3 = 0x44444444
Read Register 0	Read address 0x00	PRDATA = 0x11111111
Read Register 1	Read address 0x04	PRDATA = 0x22222222
Read Register 2	Read address 0x08	PRDATA = 0x33333333
Read Register 3	Read address 0x0C	PRDATA = 0x44444444

Design Code:

```

1  `timescale 1ns/1ps
2
3  module apb_slave(
4
5      input  logic      PCLK,
6      input  logic      PRESETn,
7
8      input  logic [31:0] PADDR,
9      input  logic [31:0] PWDATA,
10
11     input  logic      PSEL,
12     input  logic      PENABLE,
13     input  logic      PWRITE,
14
15     output logic [31:0] PRDATA,
16     output logic      PREADY,
17     output logic      PSLVERR
18
19 );
20
21 logic [31:0] reg0;
22 logic [31:0] reg1;
23 logic [31:0] reg2;
24 logic [31:0] reg3;
25
26 //Write Logic
27
28 always_ff @(posedge PCLK or negedge PRESETn)
29 begin
30
31     if(!PRESETn)
32     begin
33         reg0 <= 32'd0;
34         reg1 <= 32'd0;
35         reg2 <= 32'd0;
36         reg3 <= 32'd0;
37     end
38
39     else if(PSEL && PENABLE && PWRITE)
40     begin
41
42         case(PADDR)
43
44             32'h00000000 : reg0 <= PWDATA;|

```

```

45         32'h00000004 : reg1 <= PWDATA;
46         32'h00000008 : reg2 <= PWDATA;
47         32'h0000000C : reg3 <= PWDATA;
48
49     endcase
50
51 end
52
53 end
54
55 //Read Logic
56
57 always_comb
58 begin
59     PRDATA = 32'd0;
60
61     if(PSEL && !PWRITE)
62     begin
63         case(PADDR)
64
65             32'h00000000 : PRDATA = reg0;
66             32'h00000004 : PRDATA = reg1;
67             32'h00000008 : PRDATA = reg2;
68             32'h0000000C : PRDATA = reg3;
69
70             default : PRDATA = 32'd0;
71
72         endcase
73     end
74
75 end
76
77 end
78
79 assign PREADY = 1'b1;
80 assign PSLVERR = 1'b0;
81
82
83 endmodule

```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb;
4
5  logic          PCLK;
6  logic          PRESETn;
7
8  logic [31:0]   PADDR;
9  logic [31:0]   PWDATA;
10 logic [31:0]   PRDATA;
11
12 logic          PSEL;
13 logic          PENABLE;
14 logic          PWRITE;
15
16 logic          PREADY;
17 logic          PSLVERR;
18
19 //DUT
20
21 apb_slave dut(
22
23   .PCLK(PCLK),
24   .PRESETn(PRESETn),
25
26   .PADDR(PADDR),
27   .PWDATA(PWDATA),
28   .PRDATA(PRDATA),
29
30   .PSEL(PSEL),
31   .PENABLE(PENABLE),
32   .PWRITE(PWRITE),
33
34   .PREADY(PREADY),
35   .PSLVERR(PSLVERR)
36
37 );
38
39 //Clock
40
41 initial
42 begin
43     PCLK = 0;
44     forever #5 PCLK = ~PCLK;

```

```
45 end
46
47 //Waveform Dump
48
49 initial
50 begin
51     $dumpfile("dump.vcd");
52     $dumpvars(0, tb);
53 end
54
55 //APB Write Task
56
57 task apb_write(input [31:0] addr,
58               input [31:0] data);
59
60 begin
61     @(posedge PCLK);
62
63     PADDR    <= addr;
64     PWDATA   <= data;
65     PWRITE   <= 1;
66     PSEL     <= 1;
67     PENABLE  <= 0;
68
69     @(posedge PCLK);
70
71     PENABLE  <= 1;
72
73     @(posedge PCLK);
74
75     PSEL     <= 0;
76     PENABLE  <= 0;
77     PWRITE   <= 0;
78
79     $display("WRITE Address=%h Data=%h",addr,data);
80
81 end
82
83
84 endtask
85
```



```

86 //APB Read Task
87
88 task apb_read(input [31:0] addr);
89
90 begin
91
92     @(posedge PCLK);
93
94     PADDR    <= addr;
95     PWRITE   <= 0;
96     PSEL     <= 1;
97     PENABLE  <= 0;
98
99     @(posedge PCLK);
100
101     PENABLE  <= 1;
102
103     @(posedge PCLK);
104
105     $display("READ Address=%h Data=%h",addr,PRDATA);
106
107     PSEL     <= 0;
108     PENABLE  <= 0;
109
110 end
111
112 endtask
113
114 //Test Sequence
115
116 initial
117 begin
118
119     PRESETn = 0;
120
121     PADDR    = 0;
122     PWDATA   = 0;
123
124     PSEL     = 0;
125     PENABLE  = 0;
126     PWRITE   = 0;
127
128     #20;
129

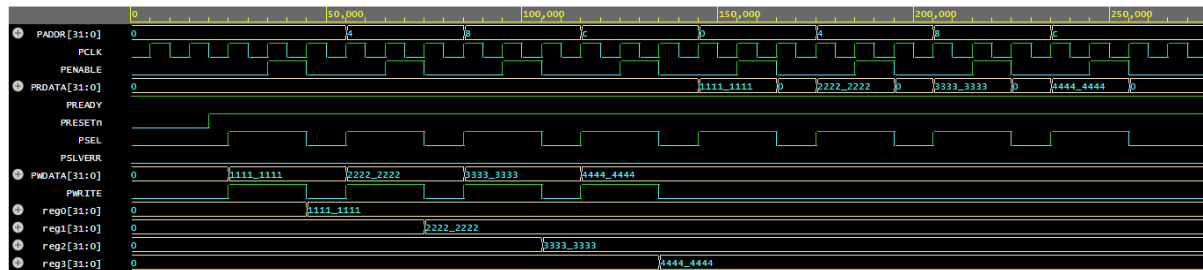
```

```

130     PRESETn = 1;
131
132     //Write Tests
133
134     apb_write(32'h00000000,32'h11111111);
135     apb_write(32'h00000004,32'h22222222);
136     apb_write(32'h00000008,32'h33333333);
137     apb_write(32'h0000000C,32'h44444444);
138
139     //Read Tests
140
141     apb_read(32'h00000000);
142     apb_read(32'h00000004);
143     apb_read(32'h00000008);
144     apb_read(32'h0000000C);
145
146     #20;
147
148     $finish;
149
150 end
151
152 endmodule

```

Simulation Result:



The simulation verifies the correct operation of the APB slave peripheral. After reset, all registers are initialized to zero. During each write transaction, the corresponding register is updated with the value present on PWDATA. During each read transaction, the stored data is correctly returned on PRDATA. The PREADY signal remains asserted throughout the simulation, indicating that the slave is always ready to complete transactions, while PSLVERR remains deasserted, confirming that no bus errors occurred. The observed waveform matches the expected results defined in the testbench.

Week 2 LAB Report

Lab Task:

Objective:

Implement a simple AHB-Lite master/slave and test read/write transactions.

Introduction:

The Advanced High-performance Bus Lite (AHB-Lite) is a high-speed bus protocol developed by ARM as part of the AMBA (Advanced Microcontroller Bus Architecture) specification. It is designed for efficient communication between processors, memories, and high-speed peripherals in single-master systems. In this experiment, a simple AHB-Lite master/slave system is implemented and simulated to verify read and write transactions.

AHB-Lite Protocol Overview:

AHB-Lite is a simplified version of the AMBA AHB protocol intended for single-master systems. It supports high-performance data transfers through pipelined operations and provides a simple interface for connecting processors with memories and peripherals.

The main signals used in this experiment are:

- HCLK: System clock signal.
- HRESETn: Active-low reset signal.
- HADDR: Address bus used to select registers.
- HWDATA: Write data bus.
- HRDATA: Read data bus.
- HWRITE: Indicates whether the current transaction is a read or write operation.
- HTRANS: Specifies the type of transfer (IDLE, NONSEQ, etc.).
- HSEL: Slave select signal.
- HREADYOUT: Indicates that the slave is ready to complete the transfer.

In this design, the slave always remains ready ($HREADYOUT = 1$) and supports simple register read and write operations.

Design Implementation:

The AHB-Lite slave was designed as a register-mapped peripheral consisting of four internal 32-bit registers.

Register Address Mapping:

Address	Register
0x00	reg0
0x04	reg1
0x08	reg2
0x0C	reg3

Design Steps:

1. Define all AHB-Lite interface signals.
2. Create four 32-bit internal registers (reg0, reg1, reg2, and reg3).
3. Implement reset logic to initialize all registers to zero when HRESETn is asserted low.
4. Implement write logic to store HWDATA into the selected register whenever:
HSEL = 1
HWRITE = 1
HTRANS = NONSEQ (2'b10) or SEQ (2'b11)
5. Implement combinational read logic to place register contents on HRDATA during read operations.
6. Keep HREADYOUT permanently asserted to indicate zero wait-state transfers.

Test Bench:

The testbench acts as a simple AHB-Lite master by generating clock, reset, and transaction control signals. Separate tasks were created for write and read operations to simplify verification.

Test Case 1 (Reset Verification):

- Apply active-low reset (HRESETn = 0).
- Expected Result: All registers should become 0x00000000.

Test Case 2 (Write Transactions):

The following values were written into the slave registers:

Address	Data Written
0x00	0xAAAAAAAA
0x04	0xBBBBBBBB
0x08	0xCCCCCCCC
0x0C	0xDDDDDDDD

- Expected Result: Each register should store the corresponding data value.

Test Case 3 (Read Transactions):

- Read operations were performed from all four addresses.
- Expected Result: The values returned on HRDATA should match the previously written data.

Design Code:

```

1  `timescale 1ns/1ps
2
3  module ahb_slave(
4
5      input  logic      HCLK,
6      input  logic      HRESETn,
7
8      input  logic [31:0] HADDR,
9      input  logic [31:0] HWDATA,
10     input  logic [1:0]  HTRANS,
11     input  logic      HWRITE,
12     input  logic      HSEL,
13
14     output logic [31:0] HRDATA,
15     output logic      HREADYOUT
16 );
17
18 logic [31:0] reg0;
19 logic [31:0] reg1;
20 logic [31:0] reg2;
21 logic [31:0] reg3;
22
23 // Write Logic
24
25 always_ff @(posedge HCLK or negedge HRESETn)
26 begin
27
28     if(!HRESETn)
29     begin
30         reg0 <= 32'd0;
31         reg1 <= 32'd0;
32         reg2 <= 32'd0;
33         reg3 <= 32'd0;
34     end
35
36     else if(HSEL &&
37             HWRITE &&
38             (HTRANS == 2'b10 || HTRANS == 2'b11))
39     begin
40
41         case(HADDR)
42
43             32'h00000000 : reg0 <= HWDATA;
44             32'h00000004 : reg1 <= HWDATA;

```

```
45         32'h00000008 : reg2 <= HWDATA;
46         32'h0000000C : reg3 <= HWDATA;
47
48     endcase
49
50 end
51
52 end
53
54 // Read Logic
55
56 always_comb
57 begin
58
59     HRDATA = 32'd0;
60
61     if(HSEL && !HWRITE)
62     begin
63
64         case(HADDR)
65
66             32'h00000000 : HRDATA = reg0;
67             32'h00000004 : HRDATA = reg1;
68             32'h00000008 : HRDATA = reg2;
69             32'h0000000C : HRDATA = reg3;
70
71             default : HRDATA = 32'd0;
72
73         endcase
74
75     end
76
77 end
78
79 assign HREADYOUT = 1'b1;
80
81 endmodule
```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb;
4
5  logic      HCLK;
6  logic      HRESETn;
7
8  logic [31:0] HADDR;
9  logic [31:0] HWDATA;
10 logic [31:0] HRDATA;
11
12 logic [1:0]  HTRANS;
13
14 logic      HWRITE;
15 logic      HSEL;
16 logic      HREADYOUT;
17
18 // DUT
19
20 ahb_slave dut(
21
22   .HCLK(HCLK),
23   .HRESETn(HRESETn),
24
25   .HADDR(HADDR),
26   .HWDATA(HWDATA),
27   .HRDATA(HRDATA),
28
29   .HTRANS(HTRANS),
30   .HWRITE(HWRITE),
31   .HSEL(HSEL),
32
33   .HREADYOUT(HREADYOUT)
34 );
35
36 // Clock Generation
37
38 initial
39 begin
40     HCLK = 0;
41     forever #5 HCLK = ~HCLK;
42 end
43
44

```

```

45 // Waveform Dump
46
47 initial
48 begin
49     $dumpfile("dump.vcd");
50     $dumpvars(0,tb);
51 end
52
53 // AHB Write Task
54
55 task ahb_write(
56     input [31:0] addr,
57     input [31:0] data
58 );
59
60 begin
61
62     @(posedge HCLK);
63
64     HADDR  <= addr;
65     HWDATA <= data;
66
67     HWRITE <= 1'b1;
68     HSEL   <= 1'b1;
69
70     // NONSEQ Transfer
71     HTRANS <= 2'b10;
72
73     @(posedge HCLK);
74
75     HSEL   <= 1'b0;
76     HWRITE <= 1'b0;
77     HTRANS <= 2'b00;
78
79     $display("WRITE Addr=%h Data=%h",
80             addr,data);
81
82 end
83
84 endtask
85
86 // AHB Read Task
87

```



```

88 task ahb_read(
89     input [31:0] addr
90 );
91
92 begin
93
94     @(posedge HCLK);
95
96     HADDR  <= addr;
97
98     HWRITE <= 1'b0;
99     HSEL   <= 1'b1;
100
101     HTRANS <= 2'b10;
102
103     @(posedge HCLK);
104
105     $display("READ Addr=%h Data=%h",
106             addr, HRDATA);
107
108     HSEL   <= 1'b0;
109     HTRANS <= 2'b00;
110
111 end
112
113 endtask
114
115 // Test Cases
116
117 initial
118 begin
119
120     HRESETn = 0;
121
122     HADDR    = 0;
123     HWDATA   = 0;
124
125     HSEL     = 0;
126     HWRITE   = 0;
127     HTRANS   = 0;
128
129     #20;
130

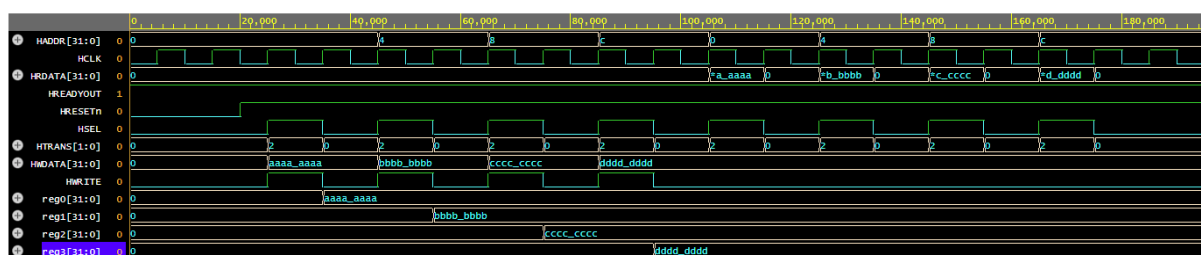
```

```

131     HRESETn = 1;
132
133     // Write Transactions
134
135     ahb_write(
136         32'h00000000,
137         32'hAAAAAAAA
138     );
139
140     ahb_write(
141         32'h00000004,
142         32'hBBBBBBBB
143     );
144
145     ahb_write(
146         32'h00000008,
147         32'hCCCCCCCC
148     );
149
150     ahb_write(
151         32'h0000000C,
152         32'hDDDDDDDD
153     );
154
155     // Read Transactions
156
157     ahb_read(32'h00000000);
158     ahb_read(32'h00000004);
159     ahb_read(32'h00000008);
160     ahb_read(32'h0000000C);
161
162     #20;
163
164     $finish;
165
166 end
167
168 endmodule

```

Simulation Result:



The waveform verifies the successful operation of the AHB-Lite slave.

1. After reset, all internal registers were initialized to 0x00000000.
2. During write transactions, HSEL and HWRITE were asserted and HTRANS entered the NONSEQ state (2'b10). The corresponding registers successfully stored the values present on HWDATA.
3. During read transactions, HWRITE was deasserted and the previously stored data appeared correctly on HRDATA.

4. HREADYOUT remained asserted throughout the simulation, indicating that the slave completed all transfers without wait states.

The simulation results matched the expected behavior specified in the testbench, confirming the correct implementation of the AHB-Lite read and write operations.